01

Среда выполнения Node.js

Импорт каталога проката — потоки, проверка данных и честные ошибки

Версия	0.2.0 от 2026-10-25
Авторы	Евгений Надточеев
Время	чтение ~90 мин, практика ~11 ч
Код примеров	папка examples/week-01 стартового репозитория
Лицензия	© Евгений Надточеев. Все права защищены.

Содержание

- Перед началом
- Результаты обучения
- Задача проката
- Теория
  - Node.js и браузер: одна модель выполнения
  - Цикл событий
  - Блокировка и как её увидеть
  - Модули и TypeScript без сборки
  - Потоки и обратное давление
  - Проверка данных во время выполнения
  - Деньги: целые копейки
  - Атомарная замена файла
  - Ошибки, коды выхода и вывод
  - Линтер
  - HTTP на node:http и мягкая остановка
- Разобранные примеры
  - Задержка цикла событий
  - Порядок колбэков
  - Обратное давление
  - Сводка по журналу возвратов
  - Атомарная запись небольшого файла
  - Сервер и мягкая остановка
- Практикум
  - Как работать с ИИ-ассистентом в курсе
  - Упражнения
  - Основное задание
- Типовые ошибки и диагностика
- Самопроверка
- Вопросы собеседования
- Что читать дальше
- Глоссарий
- Журнал изменений
- Приложение. Ответы на самопроверку

Перед началом

Методичка для разработчика, который уверенно пишет на JavaScript: промисы, `async` / `await`, классы, модули. TypeScript нужен на уровне чтения типов, объединений и дженериков. Терминал, Git и запуск Node.js описаны в методичке 00.

Что понадобится:

- Node.js 22.18 или новее, лучше актуальная LTS (26). Версию покажет `node -v`.
- npm, редактор с поддержкой TypeScript, терминал с `curl`.
- Около 90 минут на чтение и 11 часов на практику: 7 часов на реализацию вместе с упражнениями (сами упражнения — около двух часов), 2 на тесты и 2 на исправления после проверки.

Все листинги приведены целиком. Подпись над листингом — путь к файлу в репозитории примеров курса; там же лежат тесты, которые эти листинги проходят. Примеры решают соседние задачи, а не основное задание недели: код задания вы пишете сами.

Результаты обучения

После этой методички вы сможете:

1. Объяснить, почему синхронная работа задерживает все запросы процесса, и измерить эту задержку (К01).
2. Назвать порядок выполнения синхронного кода, микрозадач, таймеров и `setImmediate` и показать, на какой порядок нельзя опираться (К01).
3. Обработать файл любого размера в ограниченной памяти: читать построчно и писать с учётом обратного давления (К02).
4. Принимать внешние данные как `unknown`, проверять их схемой во время выполнения и писать сообщения, по которым сотрудник найдёт ошибку (К03).
5. Хранить деньги целым числом копеек и отклонять дробные суммы (К10).
6. Заменять файл атомарно и различать ошибку данных и сбой ввода-вывода кодом выхода, а каждый такой случай закреплять тестом на `node:test` (К03, К19).
7. Поднять HTTP-сервер на `node:http` и останавливать его по SIGTERM без потери начатых запросов (К04).
8. Пользоваться ИИ-ассистентом так, чтобы решение оставалось вашим: проверять предложенное запуском и тестами и раскрывать существенную помощь в MR (К24).

Задача проката

Пункт проката инструментов переезжает со старой учётной системы на новый сервис. Старая система умеет только выгрузку: файл JSONL, в котором каждая строка — одна позиция каталога.

```
{"id":"drill-bosch-gsb-13","name":"Дрель ударная Bosch GSB 13 RE","category":"drills",
 "dailyPrice":45000,"deposit":500000,"currency":"RUB","specs":{"powerW":600}}
```

Здесь выгрузка перенесена на две строки ради ширины страницы; в файле это одна строка. Цена за сутки — 450 рублей, залог — 5 000 рублей, обе суммы в копейках.

Сейчас в каталоге несколько сотен позиций. Когда к сервису подключится сеть пунктов, их станут сотни тысяч, а файл — сотни мегабайт. Сотрудник запускает импорт и ждёт одного из двух исходов:

- каталог заменён целиком, и выведено, сколько позиций загружено;
- каталог не тронут, и есть список строк с понятной причиной ошибки.

Худший исход — третий, которого сотрудник не ждёт: импорт упал посередине, и на витрине в субботу утром половина каталога. Клиенты не находят инструмент, бронирования уходят конкурентам. Не лучше и «тихий» успех, когда файл не прочитался, а программа сообщила, что загрузила ноль позиций и стёрла старый каталог.

Типичные дефекты выгрузки известны заранее:

- дробные цены вроде `450.5` — следы ручного редактирования в таблице;
- повторяющиеся `id` после слияния выгрузок двух пунктов, причём не подряд;
- оборванная последняя строка, если выгрузку прервали;
- лишние поля с опечатками: `dailyprice` вместо `dailyPrice`.

Вторая часть недели — проверка живости. Сервис будет работать в контейнере, и системе запуска нужен адрес, по которому видно, что процесс отвечает. Это первый HTTP-маршрут сервиса; каталог по HTTP появится позже.

Теория

Node.js и браузер: одна модель выполнения

Node.js — это движок V8, который выполняет JavaScript, библиотека libuv, которая отвечает за ввод-вывод и цикл событий, и стандартная библиотека модулей `node:*`. В браузере те же промисы и `fetch`, но нет файловой системы, сокетов, переменных окружения и кода выхода. В Node.js нет DOM и `window`.

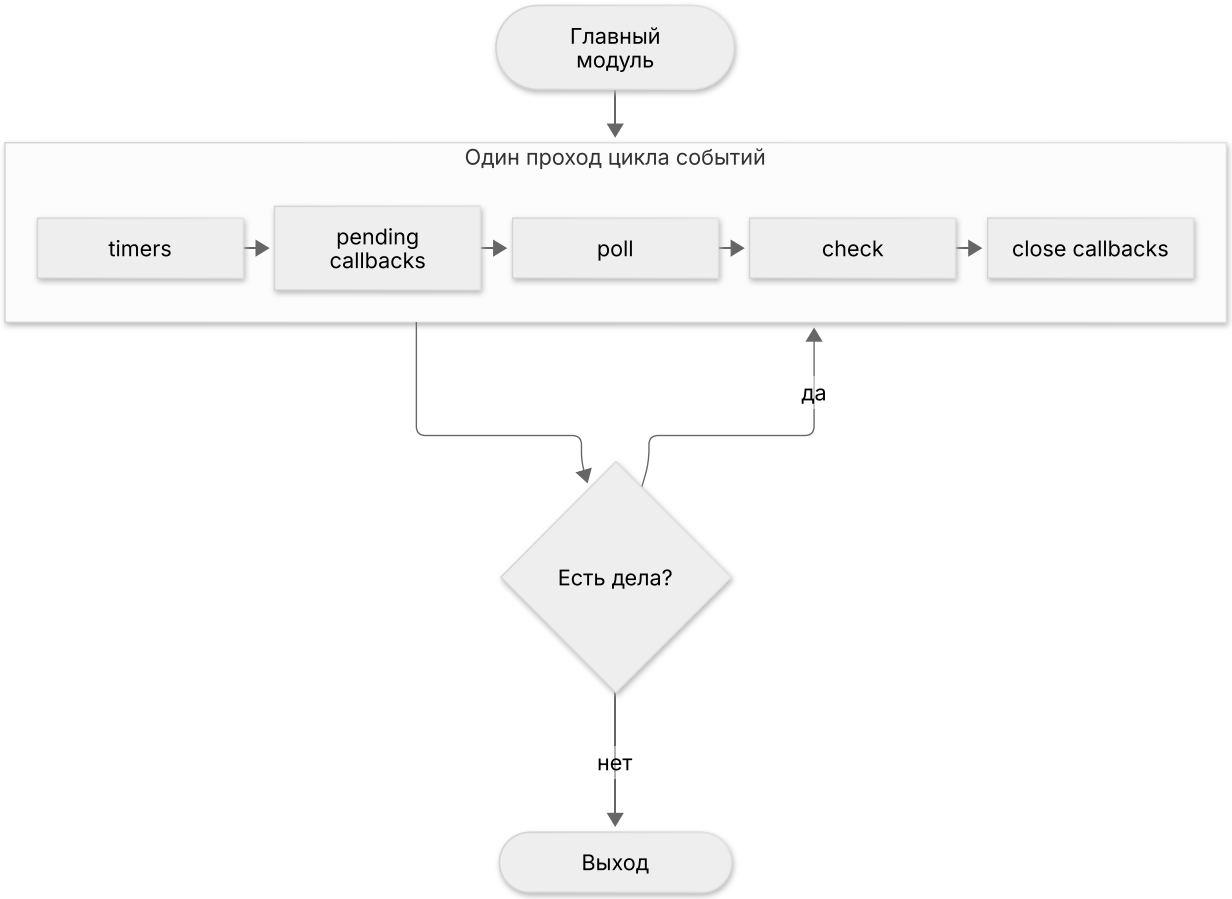
Важнее различий общее: в обоих средах ваш JavaScript выполняется в одном потоке. Чтение файла или ожидание сети выполняет операционная система или пул потоков libuv, а колбэк с результатом возвращается в этот единственный поток. Пока поток занят вашим кодом, ни один другой колбэк не выполнится.

Отсюда второе правило, которое пригодится уже на этой неделе: **процесс завершается, когда в цикле событий не осталось дел** — открытых серверов, таймеров, незавершённых операций ввода-вывода. Сервер живёт, пока слушает порт. Скрипт импорта завершается, когда

дочитал файл и закрыл всё, что открыл. Если скрипт «висит», значит, что-то осталось открытым; если завершился раньше времени, значит, вы ждали промис, который ничто не разрешит.

Цикл событий

Цикл событий проходит по фазам, и в каждой выполняет свою очередь колбэков.



- **timers** — колбэки `setTimeout` и `setInterval`, у которых истёк срок.
- **pending callbacks** — колбэки ввода-вывода, отложенные с прошлого прохода.
- **poll** — получение новых событий ввода-вывода и их колбэки; здесь процесс ждёт, если делать нечего.
- **check** — колбэки `setImmediate`.
- **close callbacks** — события закрытия, например `socket.on('close')`.

Схема упрощена. С libuv 1.45 (Node.js 20 и новее) таймеры выполняются после фазы poll, а перед входом в цикл — один раз дополнительно. Поэтому в главном модуле порядок `setTimeout(fn, 0)` и `setImmediate(fn)` не определён и зависит от загрузки машины. Внутри колбэка ввода-вывода `setImmediate` всегда срабатывает раньше таймера.

Между фазами есть ещё две очереди, которые к циклу формально не относятся:

- **очередь `process.nextTick`** — специфична для Node.js;
- **очередь микрозадач** — колбэки промисов и `queueMicrotask`.

Обе выполняются полностью после каждой операции, прежде чем цикл перейдёт дальше. Поэтому бесконечная цепочка микрозадач блокирует процесс так же, как бесконечный цикл: до таймеров и сети очередь не дойдёт.

Пример `order.ts` в разделе примеров заводит по одному колбэку каждого вида. Результаты запуска:

Где выполняется	Порядок
CommonJS (`.cts`), Node.js 22–26	sync → nextTick → promise → immediate → timeout
ES-модуль, Node.js 22 и 24	sync → promise → nextTick → …
ES-модуль без top-level `await`, Node.js 26.12	sync → nextTick → promise → …
ES-модуль после top-level `await`, Node.js 26.12	sync → promise → nextTick → …

Последние две позиции, `immediate` и `timeout`, в главном модуле могут меняться местами. Различие ES-модулей появилось, когда Node.js начал загружать большинство ES-модулей синхронно: в версии 26.12 главный модуль без top-level `await` выполняется вне очереди микрозадач. Вывод практический: **не стройте логику на порядке между `nextTick` и промисами**. Если одному действию нужно выполниться после другого, свяжите их явно через `await`.

Блокировка и как её увидеть

Сервис проката обслуживает запросы клиентов и импорт в одном процессе. Если импорт пересчитывает цены по всему каталогу синхронным циклом за две секунды, то две секунды не отвечает никто: ни клиент, ни проверка живости. Система запуска решит, что процесс завис, и перезапустит его.

Типичные источники блокировки в backend-коде:

- синхронные функции файловой системы (`readFileSync`) в обработчике запроса;
- `JSON.parse` и `JSON.stringify` больших объёмов;
- синхронная криптография (`pbkdf2Sync`, `scryptSync`);
- регулярные выражения с катастрофическим перебором;
- сортировки и вложенные циклы по всему набору данных.

Блокировку видно по задержке таймера. Пример `event-loop-lag.ts` заводит таймер на 5 мс, выполняет работу и измеряет, насколько позже срока таймер сработал. Синхронная работа на 300 мс задерживает таймер на все 300 мс. Та же работа порциями по 10 мс с `await setImmediate()` между ними задерживает его примерно на одну порцию.

Порции не ускоряют работу: она длится столько же или чуть дольше. Они делят поток с остальными задачами. Тяжёлый расчёт, который нельзя разбить, уносят в `worker_threads` или отдельный процесс. В задачах курса такого расчёта нет, и дальше эта тема не разбирается; с чего начать — в разделе «Что читать дальше». В работающем сервисе задержку цикла событий меряют гистограммой `monitorEventLoopDelay` из `node:perf_hooks` и выводят в метрики.

Модули и TypeScript без сборки

Проект курса — ES-модули: в `package.json` указано `"type": "module"`. Файлы `.ts` и `.js` загружаются как ES-модули, `.cts` и `.cjs` — как CommonJS. Расширение в относительном импорте обязательно: `import './item.ts'`, а не `import './item'`. Путь к текущему файлу дают `import.meta.filename` и `import.meta.dirname`, а не `__filename` и `__dirname`.

Node.js запускает TypeScript напрямую, удаляя аннотации типов (type stripping). С версий 22.18 и 23.6 это включено по умолчанию, с 25.2 и 24.12 считается стабильным. У подхода есть границы, и их нужно знать.

Node.js не проверяет типы. Он вырезает аннотации и выполняет то, что осталось. Ошибка типа в коде доживёт до продакшена, если не запускать `tsc`. В проекте курса это команда `npm run typecheck`, и она входит в `npm run check` вместе с линтером и тестами.

Поддерживается только стираемый синтаксис. Всё, что TypeScript превращает в JavaScript-код, вызывает ошибку `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`: `enum`, `namespace` с кодом, свойства в параметрах конструктора, `import x = require()`. В Node.js 26 флаг `--experimental-transform-types`, который это обходил, удалён. Вместо `enum` используйте объединение строковых литералов или объект с `as const`.

Импорт типа помечается словом `type`. Без него Node.js считает импорт значением и падает при загрузке:

```
SyntaxError: The requested module './t2.ts' does not provide an export named 'Item'
```

`tsconfig.json` Node.js не читает. Пути вроде `@/…` и понижение синтаксиса не работают. Файлы TypeScript внутри `node_modules` Node.js не выполняет намеренно: библиотеки публикуют JavaScript.

Настройки `tsconfig.json`, которые делают проверку типов честной именно для такого запуска:

Настройка	Зачем
`"module": "nodenext"`	Разрешение модулей как у Node.js
`allowImportingTsExtensions` и `noEmit`	Разрешают `.ts` в импортах; `tsc` только проверяет
`verbatimModuleSyntax`	Требует `import type` там, где импортируется только тип
`erasableSyntaxOnly`	Запрещает `enum` и прочее, что Node.js не умеет стирать
`strict`, `noUncheckedIndexedAccess`	`arr[i]` имеет тип `T | undefined`, как и на деле
`exactOptionalPropertyTypes`	Различает «поля нет» и «поле равно undefined»

Потоки и обратное давление

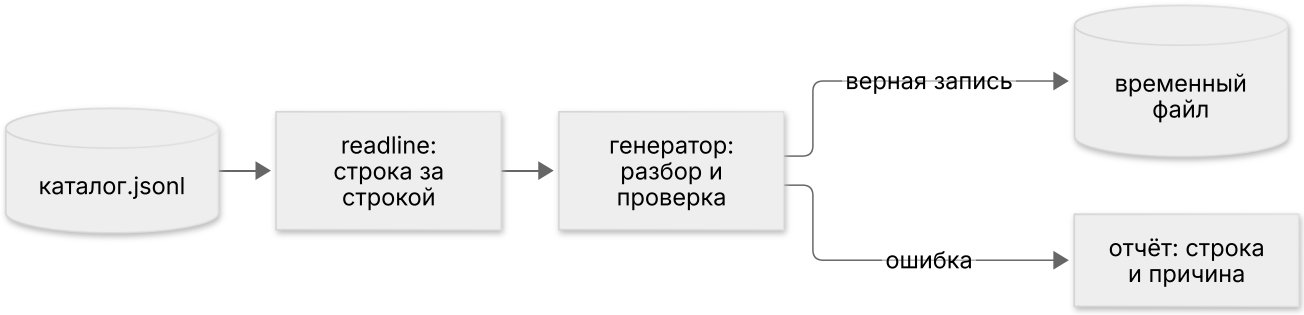
Файл на сотни мегабайт нельзя прочитать целиком: `readFile` положит его в память одной строкой, а `JSON.parse` каждой записи удвоит расход. Процесс с ограничением памяти в контейнере упадёт с `JavaScript heap out of memory`. Поток (stream) отдаёт данные порциями, и в

памяти одновременно находится только небольшой буфер.

У потока на запись (Writable) есть внутренний буфер с порогом `highWaterMark`. Метод `write()` принимает данные всегда, но возвращает `false`, если буфер заполнен выше порога. Это сигнал: остановитесь и дождитесь события `drain`. Такая обратная связь от медленного приёмника к быстрому источнику называется обратным давлением (backpressure).

Код, который игнорирует `false`, работает на маленьком файле и падает на большом: всё, что диск не успел записать, копится в памяти. В примере `backpressure.ts` 5 000 строк пишутся в медленный приёмник с буфером 1 КБ. Без ожидания `drain` в буфере скапливается больше 50 КБ — то есть весь файл. С ожиданием буфер не выходит за порог плюс одну строку.

Вручную ждать `drain` приходится редко. Функция `pipeline` из `node:stream/promises` соединяет источник, преобразования и приёмник, сама соблюдает обратное давление, передаёт ошибку любого звена и закрывает все потоки при сбое. Источником может быть асинхронный генератор:



Пока приёмник занят, `pipeline` не запрашивает у генератора следующую строку, а генератор не читает следующую строку из файла. Память ограничена буферами потоков, а не размером файла.

Для построчного чтения подходит `readline.createInterface` с циклом `for await`. Параметр `crlfDelay: Infinity` нужен для файлов с переводом строки `\r\n` из Windows: без него `\r` и `\n`, пришедшие с паузой, считаются двумя переводами строки, и нумерация строк поедет. Документация отмечает, что асинхронный итератор медленнее событий `'line'`; для импорта это несущественно.

Учтите и ещё одно: документация `readline` пишет, что ошибки входного потока в асинхронный итератор не передаются. В Node.js 22, 24 и 26 итератор такую ошибку бросает — это проверено тестом. Расхождение текста и поведения — повод закрепить нужное поведение собственным тестом, а не полагаться на формулировку.

Проверка данных во время выполнения

Типы TypeScript стираются до запуска. `JSON.parse` возвращает `any`, и присвоение его переменной типа `CatalogItem` ничего не проверяет: компилятор поверит на слово. Данные из файла, сети и переменных окружения принимайте как `unknown` и превращайте в доменный тип только после проверки.

Проверку удобно описывать схемой. В курсе используется zod 4: схема описывает ограничения, `safeParse` возвращает либо данные, либо список нарушений, а тип выводится из схемы через `z.infer`, поэтому схема и тип не расходятся.

Несколько решений, которые стоит принимать явно:

- **Лишние поля — ошибка.** `z.strictObject` отклоняет неизвестные ключи. Иначе опечатка `dailyprice` тихо потеряет цену, а обязательное `dailyPrice` окажется пропущенным.
- **`JSON.parse` перехватывайте узко.** Ловите только `SyntaxError`; любую другую ошибку пробрасывайте. `catch`, который ловит всё подряд, превратит нехватку памяти в «неверный JSON».
- **Нумеруйте строки по файлу**, включая пустые. Сотрудник откроет файл в редакторе и будет искать именно эту строку.
- **Пишите сообщения для читателя.** Встроенная локализация zod даёт «Неверный ввод: ожидалось string, получено undefined». Разработчику это понятно, сотруднику пункта проката — нет. Для отчёта импорта задайте сообщения в схеме: «name: обязательное поле», «dailyPrice: должно быть целым числом копеек».
- **Ограничивайте отчёт.** В файле на сто тысяч битых строк не нужно сто тысяч сообщений. Показывайте первые N и общее количество.

Глобальная настройка `z.config()` меняет поведение всех схем процесса, в том числе в чужих модулях. Язык сообщений лучше передавать при вызове, как в примере `returns-summary.ts`.

Деньги: целые копейки

Числа в JavaScript — двоичная плавающая точка: `0.1 + 0.2` равно `0.30000000000000004`. Такие погрешности накапливаются в суммах и скидках, а округление в разных местах кода превращает их в копейку расхождения, которую потом ищут по всей кассе за месяц. Поэтому суммы хранят целым числом минимальных единиц: 450 рублей — это `45000` копеек.

Целые числа в `number` точны до `Number.MAX_SAFE_INTEGER`, это 9 007 199 254 740 991 копейка — около 90 триллионов рублей. Для цены и залога запас огромный. `z.int()` в zod 4 принимает только безопасные целые числа.

Дробную сумму в выгрузке не округляйте, а отклоняйте. `450.5` копейки — признак того, что в источнике рубли перепутали с копейками или поправили ячейку руками. Округление спрячет ошибку источника, отказ покажет её.

Валюта хранится явно, даже если пока она одна. Поле `currency: "RUB"` стоит ничего, а без него первая же интеграция с другой валютой превратится в поиск по всему коду.

Атомарная замена файла

Если писать результат прямо в рабочий каталог, сбой посередине оставит половину файла. Сбоем может быть ошибка в данных на десятитысячной строке, нехватка места на диске или остановка процесса. Читатель каталога в этот момент увидит обрывок.

Надёжная схема из трёх шагов:

1. Писать во временный файл **в той же папке**, что и целевой.
2. Дождаться, пока данные дойдут до диска: опция `flush: true` вызывает `fsync` перед закрытием файла.
3. Переименовать временный файл поверх целевого через `rename`.

По POSIX `rename` заменяет существующий файл так, что другой процесс видит либо старую версию, либо новую, но не промежуточную. Гарантия действует только в пределах одной файловой системы. Временный файл в `/tmp` может оказаться на другом разделе, и `rename` упадёт с `EXDEV`. На Windows замена может не пройти, если целевой файл держит открытым другой процесс (`EPERM`, `EBUSY`).

Без `fsync` после сбоя питания можно получить новое имя, которое указывает на ещё не записанные данные, то есть пустой файл. Для полной устойчивости к сбою питания нужен ещё `fsync` папки после `rename`. Эта методичка его не требует, но граница гарантии должна быть вам известна.

Временный файл открывайте с флагом `wx`: открытие падает, если файл уже есть, и вы не перезапишете чужой. При любой ошибке временный файл удаляйте. Если процесс убит сигналом `SIGKILL`, удалить его не успеете. Поэтому имя временного файла должно быть узнаваемым, чтобы такие остатки было легко найти.

Ошибки, коды выхода и вывод

У программы импорта три разных исхода, и вызывающая сторона должна их различать. Сотрудник читает текст, а скрипт расписания или CI — только код выхода.

Код	Исход	Пример
0	Успех	Каталог заменён
1	Ошибка в данных	Дробная цена в строке 17
2	Неверный вызов	Не передан `--input`
3	Сбой ввода-вывода	Файл не найден, нет прав, диск заполнен

Единого стандарта кодов нет. В BSD есть `sysexits.h` со значениями 64–78, многие утилиты используют 1 и 2. Важно, чтобы коды были разными и описанными в README. Главное правило — не маскировать сбой: отсутствующий входной файл не равен пустому каталогу, а ошибка записи не равна успеху.

Ошибку ввода-вывода удобно отличать собственным классом с исходной причиной в `cause`:

```
export class ReportWriteError extends Error {
  override name = 'ReportWriteError'
}

throw new ReportWriteError(`не удалось записать отчёт: ${file}`, { cause })
```

Код выхода задавайте через `process.exitCode`, а не `process.exit()`. Запись в `process.stdout` при выводе в канал (pipe) на Linux и macOS асинхронна, и `exit()` завершает процесс, не дожидаясь её. Проверьте сами: скрипт выводит 200 000 строк и вызывает `process.exit(0)`, а медленный читатель канала получает около 3 700 строк. С `process.exitCode = 0` доходят все 200 000.

Сообщение для сотрудника — причина и место ошибки, без стека вызовов и внутренних путей. Подробности для разработчика уходят в лог. Отклонённый промис, который никто не обработал, по умолчанию завершает процесс с кодом 1 и стеком в stderr. Это правильное поведение для непредвиденной ошибки, но ожидаемые исходы должны обрабатываться явно.

Линтер

Промис, которого никто не ждёт, тесты замечают не всегда: на их данных он просто не отклоняется. Его находит линтер — `npm run lint` (что это и чем он отличается от проверки типов, рассказывает методичка 00). Так он отвечает на функцию, которая пишет отчёт и не ждёт записи:

```
import { writeFile } from 'node:fs/promises'

export function saveReport(file: string, text: string): void {
  writeFile(file, text)
}
```

```
/home/you/rental/src/cli/report.ts
  4:3  error  Promises must be awaited, end with a call to .catch, end with a call
              to .then with a rejection handler or be explicitly marked as ignored
              with the `void` operator  @typescript-eslint/no-floating-promises
```

Сообщение линтер печатает одной строкой, здесь она перенесена. `4:3` — строка 4, столбец 3; в конце — имя правила: `no-floating-promises`, висящий промис (floating promise). Правило видит выражение, которое возвращает промис, а результат выброшен: никто не ждёт его и не обработает отказ. Здесь это настоящая ошибка: функция вернётся раньше, чем отчёт записан, а при нехватке места отказ станет необработанным отказом промиса (unhandled rejection) и завершит процесс с кодом 1.

Исправлений три, и выбирают по смыслу, а не по тому, что короче:

- `await` — результат нужен дальше, а ошибка должна прервать функцию. Для отчёта верно это: функция становится `async` и возвращает `Promise<void>`;
- `.catch(…)` с обработкой — ждать незачем, но отказ есть кому разобрать. Так `shutdown` в примере сервера ловит ошибку `stop()`: сообщение в stderr и код выхода 1;
- `void` — результат намеренно не нужен, а отказа быть не может. Так помечена микрозадача в примере порядка колбэков: она только ставит отметку в массив.

`void` — не способ заглушить линтер: он говорит читателю «промис брошен намеренно». Если отказ возможен, `void` оставит его необработанным, как и раньше.

Соседнее правило `no-misused-promises` ловит async-функцию там, где вызывающий промис не ждёт: `process.once('SIGTERM', async () => …)`, обработчик запроса в `createServer`. Сигнал придёт, обработчик вернёт промис, а `process` его выбросит: отказ внутри станет необработанным. Поэтому `shutdown` в примере сервера — обычная функция, а промис остановки она обрабатывает сама.

Ещё два правила — о `catch`. `no-empty` запрещает пустой `catch {}`: ошибка исчезает без следа. Блок с одним комментарием линтер пропускает, поэтому комментарий должен объяснять, почему ошибку можно потерять, — это смотрит ревью. `preserve-caught-error` требует, чтобы новая ошибка, брошенная из `catch`, несла исходную в `cause`, как `ReportWriteError` выше.

Правило можно отключить строкой над кодом: `// eslint-disable-next-line <правило> -- причина`. Это последнее средство: сначала исправление по существу. В коде курса отключений нет.

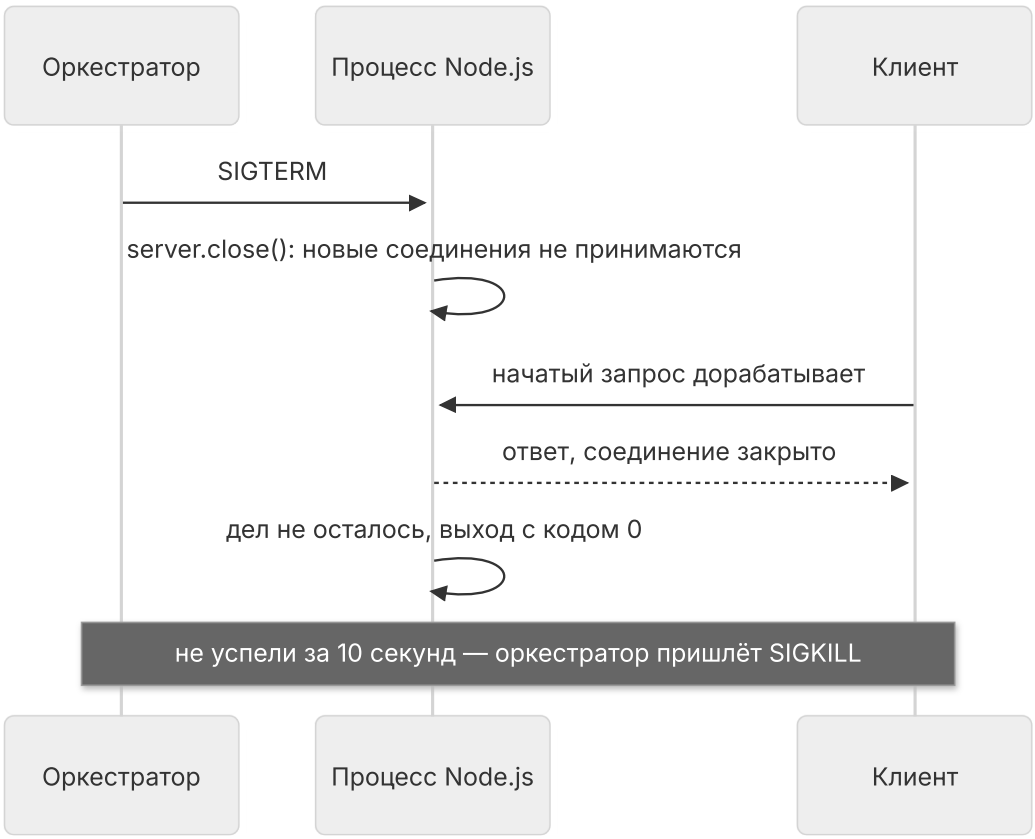
HTTP на node:http и мягкая остановка

`node:http` — низкоуровневый сервер: на каждый запрос вызывается функция с объектами запроса и ответа. Маршрутизацию, разбор тела и формат ошибок делаете вы сами; на четвёртой неделе это возьмёт на себя NestJS. Для одного маршрута проверки живости фреймворк не нужен.

Что важно даже для одного маршрута:

- Путь берите из `new URL(req.url, 'http://localhost').pathname`: строка запроса `?probe=1` не должна ломать маршрут.
- Отвечайте JSON с заголовком `content-type` и для успеха, и для ошибки. Клиент не должен угадывать формат по коду ответа.
- На неподдерживаемый метод отвечайте 405 с заголовком `Allow`, на неизвестный путь — 404. RFC 9110 требует `Allow` в ответе 405.
- Проверка живости не трогает базу и внешние сервисы. Она отвечает на вопрос «процесс жив и обслуживает цикл событий?». Готовность принимать трафик проверяется отдельно, когда появятся зависимости.

Остановка. Docker, Kubernetes и systemd останавливают процесс сигналом SIGTERM, а если он не завершился за отведённое время, присылают SIGKILL. У `docker stop` это время по умолчанию 10 секунд. Мягкая остановка — это:



Подвох в keep-alive. Клиенты держат соединение открытым после ответа, чтобы отправить по нему следующий запрос. С Node.js 19 `server.close()` закрывает соединения, которые простаивают в момент вызова. А соединение запроса, который выполнялся в момент вызова, после ответа остаётся открытым до `keepAliveTimeout` или до закрытия клиентом. В опыте с `fetch` это 3,4 секунды лишнего ожидания. Решение в примере `graceful-stop.ts`: после остановки на завершении каждого ответа вызывать `server.closeIdleConnections()`.

Порт берите из переменной окружения `PORT` и проверяйте строку, а не результат `Number()`. `Number('abc')` даёт `NaN`, и `listen` бросит `ERR_SOCKET_BAD_PORT` — это хотя бы заметно. Хуже `Number('')`: пустая строка превращается в 0, и сервер молча займёт случайный порт, по которому его никто не найдёт. Так же молча `Number()` принимает `' 80'`, `'0x50'` и `'1e3'`. Наочно `PORT=0` удобен в тестах: система выдаст свободный порт, а узнать его можно через `server.address()`.

Разобранные примеры

Задержка цикла событий

examples/week-01/event-loop-lag.ts

```ts
import { setImmediate as yieldToLoop } from 'node:timers/promises'

/** Занимает поток на ms миллисекунд — так выглядит тяжёлый синхронный расчёт. */
export function blockFor(ms: number): void {
  const end = performance.now() + ms
  while (performance.now() < end) {
    // Пустой цикл намеренно: он изображает пересчёт цен по всему каталогу.
  }
}

/** Та же работа порциями: между порциями цикл событий обслуживает таймеры и сеть. */
export async function blockInSlices(totalMs: number, sliceMs: number): Promise<void> {
  for (let done = 0; done < totalMs; done += sliceMs) {
    blockFor(Math.min(sliceMs, totalMs - done))
    await yieldToLoop()
  }
}

/** На сколько миллисекунд позже срока сработал таймер, заведённый перед работой. */
export async function measureTimerLag(
  work: () => unknown,
  delayMs = 5,
): Promise<number> {
  const started = performance.now()
  const fired = new Promise<number>((resolve) => {
    setTimeout(() => resolve(performance.now()), delayMs)
  })
  await work()
  return (await fired) - started - delayMs
}
```

Что заметить:

- `measureTimerLag(() => blockFor(300))` возвращает не меньше 250 мс, а `measureTimerLag(() => blockInSlices(300, 10))` — меньше 150 мс. Тесты проверяют именно эти границы с запасом на медленную машину.
- `setImmediate` из `node:timers/promises` — способ «уступить» поток: остаток работы продолжится после того, как цикл обслужит накопившиеся события.

Порядок колбэков

examples/week-01/order.ts

```ts
// Порядок колбэков в ES-модуле. Сравните с order.cts и с выводом на разных
// версиях Node: место nextTick относительно промиса здесь менялось.
const order: string[] = []

setTimeout(() => order.push('timeout'), 0)
setImmediate(() => order.push('immediate'))
void Promise.resolve().then(() => order.push('promise'))
process.nextTick(() => order.push('nextTick'))
order.push('sync')

setTimeout(() => console.log(order.join(' → ')), 50)
```

Файл `order.cts` содержит тот же код. Тест проверяет только устойчивые части порядка: синхронный код первым, обе микрозадачи раньше таймера и `setImmediate`, в CommonJS `nextTick` раньше промиса. Порядок `nextTick` и промиса в ES-модуле тест намеренно не проверяет. `void` перед промисом говорит линтеру и читателю, что результат брошен намеренно (раздел «Линтер»).

Обратное давление

examples/week-01/backpressure.ts

```ts
import { once } from 'node:events'
import type { Writable } from 'node:stream'

/** Так делать нельзя: write() вернул false, а мы продолжаем писать в буфер. */
export function writeIgnoringBackpressure(
  out: Writable,
  lines: Iterable<string>,
): void {
  for (const line of lines) out.write(line)
}

/** Пишет строки и ждёт drain, когда внутренний буфер потока заполнен. */
export async function writeLines(
  out: Writable,
  lines: Iterable<string>,
): Promise<void> {
  for (const line of lines) {
    // once() отклоняет промис при 'error', поэтому сбой записи не повиснет.
    if (!out.write(line)) await once(out, 'drain')
  }
}
```

Что заметить:

- Обе функции «работают» на маленьких данных. Разница видна только по объёму буфера под нагрузкой; поэтому тест меряет `writableLength`, а не результат.
- `once` из `node:events` превращает событие в промис и отклоняет его, если поток сообщил об ошибке. Ожидание `drain` на сломанном потоке не повиснет.

Сводка по журналу возвратов

Возврат инструмента клиентом записывается в журнал: какая аренда, какой инструмент и сколько удержано за повреждение. Пример считает сводку и собирает строки с ошибками — тот же приём, что нужен в импорте каталога, на другой записи и без замены файла.

examples/week-01/returns-summary.ts

```ts
import { createReadStream } from 'node:fs'
import { createInterface } from 'node:readline'
import { z } from 'zod'

// Журнал возвратов: одна строка — один возврат инструмента клиентом.
const ReturnRecord = z.strictObject({
  rentalId: z.string().min(1),
  itemId: z.string().min(1),
  damageKopecks: z.int().nonnegative(),
})

const MAX_PROBLEMS = 20
// Русские сообщения zod подключаем на вызов, а не глобально через z.config().
const ru = z.locales.ru()

export type Problem = { line: number; reason: string }
export type ReturnsSummary = {
  returns: number
  damageKopecks: number
  problems: Problem[]
  totalProblems: number
}

/** Сводка по журналу возвратов; память не растёт с размером файла. */
export async function summarizeReturns(file: string): Promise<ReturnsSummary> {
  const summary: ReturnsSummary = {
    returns: 0,
    damageKopecks: 0,
    problems: [],
    totalProblems: 0,
  }
  const problem = (line: number, reason: string) => {
    summary.totalProblems += 1
    if (summary.problems.length < MAX_PROBLEMS) summary.problems.push({ line, reason })
  }

  const lines = createInterface({
    input: createReadStream(file, { encoding: 'utf8' }),
    crlfDelay: Infinity,
  })
  let lineNo = 0
  for await (const text of lines) {
    lineNo += 1
    if (text.trim() === '') continue

    let raw: unknown
    try {
      raw = JSON.parse(text)
    } catch (error) {
      if (!(error instanceof SyntaxError)) throw error
      problem(lineNo, 'неверный JSON')
```

```
      continue
    }

    const parsed = ReturnRecord.safeParse(raw, { error: ru.localeError })
    if (!parsed.success) {
      const issue = parsed.error.issues[0]
      const field = issue?.path.join('.') || 'запись'
      problem(lineNo, `${field}: ${issue?.message ?? 'неизвестная ошибка'}`)
      continue
    }
    summary.returns += 1
    summary.damageKopecks += parsed.data.damageKopecks
  }
  return summary
}
```

На файле с четырьмя дефектами пример выдаёт такой отчёт:

```
строка 2: неверный JSON
строка 3: damageKopecks: Слишком маленькое значение: ожидалось, что number будет >=0
строка 4: itemId: Неверный ввод: ожидалось string, получено undefined
строка 5: запись: Нераспознанный ключ: "note"
```

Что заметить:

- Пустые строки пропускаются, но номер строки всё равно растёт.
- Сообщения локализованы, но написаны для разработчика. Для отчёта сотруднику сравните их с «itemId: обязательное поле» — так их стоит писать в задании.
- Пример не ищет дубли. В импорте каталога для этого нужно помнить уже встреченные `id`, и эта память растёт с каталогом. Поэтому у импорта есть объявленный предел числа позиций.

Атомарная запись небольшого файла

examples/week-01/write-atomic.ts

```ts
import { randomBytes } from 'node:crypto'
import { rename, rm, writeFile } from 'node:fs/promises'
import path from 'node:path'

/**
 * Заменяет файл целиком или не трогает его вовсе. Подходит для небольших
 * данных, которые уже лежат в памяти: прайс-лист, настройки, отчёт.
 */
export async function writeFileAtomic(target: string, data: string): Promise<void> {
  // Та же папка: rename атомарен только в пределах одной файловой системы.
  const tmp = path.join(
    path.dirname(target),
    `.${path.basename(target)}.tmp-${randomBytes(6).toString('hex')}`,
  )
  try {
    // wx: не перезаписать чужой временный файл; flush: fsync до rename.
    await writeFile(tmp, data, { flag: 'wx', flush: true })
    await rename(tmp, target)
  } catch (error) {
    await rm(tmp, { force: true })
    throw error
  }
}
```

Что заметить:

- Здесь данные целиком в памяти. В импорте каталога их нельзя накапливать: временный файл пишется потоком (`createWriteStream` с теми же `flags: 'wx'` и `flush: true`), а `rename` выполняется, только если весь вход оказался верным.
- `rm` с `force: true` не падает, если временный файл так и не был создан.
- Ошибка пробрасывается дальше: решать, что сказать пользователю, будет вызывающий код.

Сервер и мягкая остановка

examples/week-01/version-server.ts

```ts
import { createServer, type Server } from 'node:http'

/** Сервер с одним маршрутом: какая версия сервиса сейчас запущена. */
export function createVersionServer(version: string): Server {
  return createServer((req, res) => {
    const { pathname } = new URL(req.url ?? '/', 'http://localhost')
    const found = pathname === '/version' && req.method === 'GET'
    const body = JSON.stringify(found ? { version } : { error: 'not_found' })
    res.writeHead(found ? 200 : 404, {
      'content-type': 'application/json; charset=utf-8',
      'content-length': Buffer.byteLength(body),
    })
    res.end(body)
  })
}
```

examples/week-01/graceful-stop.ts

```ts
import type { Server } from 'node:http'

/**
 * Готовит сервер к мягкой остановке и возвращает функцию остановки:
 * новые соединения не принимаются, начатые запросы дорабатывают.
 */
export function prepareStop(server: Server): () => Promise<void> {
  let stopping = false
  // close() закрывает только соединения, которые простаивают в момент вызова.
  // Keep-alive соединение запроса, начатого раньше, иначе живёт до keepAliveTimeout.
  server.on('request', (_req, res) => {
    res.on('finish', () => {
      if (stopping) server.closeIdleConnections()
    })
  })
  return () =>
    new Promise<void>((resolve, reject) => {
      stopping = true
      server.close((error) => (error ? reject(error) : resolve()))
    })
}
```

examples/week-01/run-version-server.ts

```ts
import type { AddressInfo } from 'node:net'
import { createVersionServer } from './version-server.ts'
import { prepareStop } from './graceful-stop.ts'

const rawPort = process.env.PORT ?? '3000'
// Number() молча принял бы '' (это 0, то есть случайный порт), ' 80', '0x50' и '1e3'.
const port = /^\d{1,5}$/.test(rawPort) ? Number(rawPort) : Number.NaN
if (!(port <= 65_535)) {
  process.stderr.write(`PORT должен быть целым числом 0–65535: «${rawPort}»\n`)
  process.exit(2)
}

const server = createVersionServer(process.env.APP_VERSION ?? 'dev')
const stop = prepareStop(server)
server.listen(port, () => {
  const { port: actual } = server.address() as AddressInfo
  process.stdout.write(`${JSON.stringify({ msg: 'listening', port: actual })}\n`)
})

function shutdown(signal: NodeJS.Signals): void {
  process.stdout.write(`${JSON.stringify({ msg: 'shutdown', signal })}\n`)
  stop().catch((error: unknown) => {
    process.stderr.write(`остановка не удалась: ${String(error)}\n`)
    process.exitCode = 1
  })
  // Страховка от зависшего клиента; unref() не держит процесс, если всё закрылось.
  setTimeout(() => process.exit(1), 10_000).unref()
}

process.once('SIGTERM', shutdown)
process.once('SIGINT', shutdown)
```

Что заметить:

- `process.exit(2)` при неверном порту допустим: до этого момента ничего не выведено в канал и не открыто, терять нечего.
- Проверяется строка из окружения, а не результат `Number()`. Тесты запускают сервер с `PORT` равным `''`, `'abc'`, `' 80'`, `'0x50'` и `'70000'` и ждут код 2.
- После SIGTERM процесс не вызывает `exit()` при успехе. Он закрывает сервер и завершается сам, когда в цикле событий не остаётся дел. Тест проверяет код выхода 0 и время остановки.
- Таймер страховки снят с учёта через `unref()`: он сработает, только если что-то держит процесс дольше 10 секунд.

Практикум

Как работать с ИИ-ассистентом в курсе

ИИ-ассистент на курсе разрешён как рабочий инструмент. Отвечаете за код, тесты и решения вы: на ревью и на защите попросят объяснить и изменить любую строку.

Что можно: спрашивать, как устроен API и почему падает тест, просить разобрать сообщение об ошибке, предложить тестовые случаи или проверить текст README. Задание недели целиком ассистенту лучше не отдавать: недели опираются друг на друга, и пропущенный приём первой недели обойдётся дороже на пятой.

Как проверять предложенное:

- запуском: ассистент уверенно описывает устаревшее поведение и опции, которых нет, — поведение Node.js, базы или брокера проверяется на вашей версии;
- документацией из раздела «Что читать дальше», а не пересказом;
- тестом: правило, которое ассистент «учёл», должно ронять тест, если его убрать;
- чтением: строку, которую не можете объяснить, перепишите сами или удалите.

Секреты, `.env` и журналы с личными данными ассистенту не отправляйте.

Существенную помощь раскрывайте в разделе «Помощь ИИ» описания MR: что сделал ассистент, что вы, как проверили и где он ошибся. Существенная — когда решение, запрос, тест или текст ассистента вошли в проект почти без изменений или его объяснение определило ваше решение; автодополнение имени переменной — нет. Заявленная и понятая помощь оценку не снижает. К защите эти записи соберутся в раскрытие помощи ИИ (методичка 20).

Упражнения

1. Предскажите порядок (15 минут). Запишите на бумаге, что выведет код, и только потом запустите его как `.mjs` и как `.cjs`:

```
setTimeout(() => console.log('A'), 0)
Promise.resolve().then(() => {
  console.log('B')
  process.nextTick(() => console.log('C'))
  Promise.resolve().then(() => console.log('D'))
})
setImmediate(() => console.log('E'))
console.log('F')
```

Объясните каждую позицию: почему `D` выводится раньше `C` и какие две буквы могут поменяться местами от запуска к запуску.

2. Измерьте блокировку (30 минут). Добавьте в сервер версии маршрут `/reprice`, который вызывает `blockFor(2000)`. В одном терминале запросите `/reprice`, в другом сразу после этого — `/version` с замером времени:

```
curl -s -o /dev/null -w '%{time_total}\n' http://127.0.0.1:3000/version
```

Замените `blockFor` на `blockInSlices(2000, 10)` и повторите. Запишите оба времени и объясните разницу.

3. Обратное давление на диске (20 минут). Напишите скрипт, который пишет два миллиона строк отчёта в файл через `createWriteStream`, сначала игнорируя `write()`, потом через `writeLines`. Сравните пиковый `process.memoryUsage().rss`.

4. Сообщения для человека (20 минут). Перепишите схему журнала возвратов так, чтобы отчёт выглядел как «itemId: обязательное поле» и «damageKopecks: не может быть отрицательным». Подсказка: параметр `error` у схемы поля.

5. Обрезанный вывод (15 минут). Повторите опыт с 200 000 строк и `process.exit(0)`. Передайте вывод медленному читателю `node script.mjs | (sleep 1; wc -l)`. Затем замените `exit()` на `exitCode`.

6. Что останется после сбоя (20 минут). Вставьте в `writeFileAtomic` паузу на 10 секунд между `writeFile` и `rename` и завершите процесс через `kill -9`. Какие файлы остались в папке? Что увидит программа, которая читает целевой файл? Как найти и убрать остатки?

Основное задание

Напишите CLI импорта каталога и сервер проверки живости по требованиям ниже. В вашем проекте уже есть инструменты, конфигурация TypeScript и файлы-образцы. MR «Материалы недели 1» добавляет заглушки `src/cli/import-catalog.ts` и `src/http/server.ts` и публичные тесты `test/week-01`: они падают, пока заглушки не заменены вашей реализацией.

Вызов: `node src/cli/import-catalog.ts --input <файл.jsonl> --output <каталог.jsonl>`.

Входной файл — UTF-8, одна запись JSON на строку, переводы строк `\n` или `\r\n`. Пустые строки и строки из пробелов пропускаются, но учитываются в нумерации.

Поле	Тип и ограничения
`id`	Строка `^[a-z0-9][a-z0-9-]{1,63}$`, уникальна во всём файле
`name`	Строка, после нормализации от 1 до 200 символов
`category`	Строка от 1 до 64 символов
`daily-Price`	Целое число копеек, не меньше 0
`deposit`	Целое число копеек, не меньше 0
`currency`	Только `"RUB"`
`specs`	Необязательный объект; значения — строки, числа или `true` / `false`; если поля нет, в результате `{}`

Других полей быть не должно. Нормализация: в `name` и `category` удаляются пробелы по краям, а в `name` любые последовательности пробельных символов заменяются одним пробелом.

Результат — нормализованный JSONL в порядке входа. Целевой файл заменяется, только если весь вход верен. При любой ошибке прежний файл остаётся байт в байт, временных файлов не остаётся.

Ошибки данных выводятся в stderr в виде `строка N: поле: причина`. Причину формулируйте для сотрудника. Повтор `id` указывает на строку первого появления. Вход без единой записи — ошибка: пустой импорт стёр бы каталог. Выводятся первые 50 ошибок и строка с числом остальных. В конце — «Каталог не изменён.»

Объявленные пределы: не больше 100 000 позиций и не больше 64 КБ на строку. Превышение — ошибка данных, а не падение процесса. Граница этого обещания: `readline` отдаёт строку только целиком, поэтому длина проверяется, когда строка уже в памяти. Строка в десятки мегабайт без перевода строки при куче 32 МБ уронит процесс раньше проверки. Защита от неё — предел, который срабатывает раньше, чем строка собрана: подсчёт байтов по ходу чтения своим разбором потока вместо `readline`. Предел размера файла по `stat` помогает, только если он меньше памяти процесса. В задании такая защита не требуется.

Коды выхода: 0 — успех, 1 — ошибки данных, 2 — неверный вызов, 3 — сбой ввода-вывода (нет файла, нет прав, не удалось записать).

Память: обработка не зависит от размера файла. Файл около 80 МБ из строк обычной длины должен импортироваться при `node --max-old-space-size=32`.

Сервер проверки живости: `node src/http/server.ts`, порт из `PORT` (по умолчанию 3000; неверное значение — код 2 без запуска сервера). `GET` и `HEAD /health` — 200 и `{"status":"ok"}`; другие методы на `/health` — 405 с `Allow: GET, HEAD`; остальные пути — 404 и `{"error":"not_found"}`. По SIGTERM сервер останавливается мягко и завершается с кодом 0.

Тесты на `node:test` покрывают как минимум:

- успешный импорт и нормализацию;

- неверный JSON, пропущенное поле, дробную и отрицательную сумму, лишнее поле — каждый случай с номером строки;
- дубль `id` не в соседних строках;
- сохранение прежнего каталога при ошибке в середине и при пустом входе;
- отсутствие входного файла и невозможность записи — код 3;
- импорт большого файла при ограниченной куче;
- все три ответа сервера.

Критерии готовности:

- `npm run check` — типы, линтер и тесты — проходит на чистой копии репозитория по инструкции из README.
- Ни один `catch` не глотает ошибку молча. Пустой `catch` и промис без обработки находит `npm run lint`; `catch`, который ловит всё подряд и подменяет исход, ищет ревью.
- README описывает вызов, коды выхода и пределы.
- В решении нет `readFile` входного файла целиком и накопления всех записей в массиве.
- В разделе «Помощь ИИ» описания MR раскрыта существенная помощь ассистента или написано, что её не было.

Проверка себя перед сдачей:

```
npm ci && npm run check
node src/cli/import-catalog.ts --input fixtures/catalog-broken.jsonl \
  --output /tmp/catalog.jsonl; echo "код выхода: $?"
PORT=3000 node src/http/server.ts &
SERVER_PID=$!
until curl -sf 'http://127.0.0.1:3000/health' >/dev/null; do sleep 0.2; done
curl -si 'http://127.0.0.1:3000/health'
kill "$SERVER_PID"
wait "$SERVER_PID"; echo "код: $?"          # 0 после мягкой остановки
```

Сервер начинает принимать соединения не сразу после `&`, поэтому запросы идут после цикла ожидания. Если сервер не стартует, цикл не кончится: прервите его `Ctrl+C` и посмотрите вывод сервера.

Доверенная проверка запускает свои тесты на своих данных. Сверяются номер строки, имя поля и код выхода; текст причины остаётся на ваше усмотрение.

Типовые ошибки и диагностика

Симптом	Причина	Как найти	Как исправить
`JavaScript heap out of memory` на большом файле	Файл читается целиком, записи копятся в массиве или игнорируется `false` от `write()`	Запустить с `--max-old-space-size=32` на файле 80 МБ	Построчное чтение и `pipeline`
Тест: «Promise resolution is still pending but the event loop has already resolved»	Код ждёт промис, который ничто не разрешит, а других дел в цикле нет	Логи вокруг каждого `await`; найти последний сработавший	Убрать ожидание, которое не может завершиться (см. строку ниже)
Импорт зависает на закрытии файла	Поток создан из `FileHandle` с `autoClose: false`, затем `await handle.close()`	`close()` у `FileHandle` ждёт завершения операций, а поток не отпускает дескриптор	Отдать дескриптор потоку: `createWriteStream(path, { flags: 'wx', flush: true })`
Номера строк сдвинуты	Пустые строки не считаются или `\r\n` даёт лишние строки	Сравнить с номером в редакторе на файле из Windows	Считать каждую строку; `crlfDelay: Infinity`
`ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`	`enum`, `namespace`, свойства в параметрах конструктора	Текст ошибки указывает на строку	Объединение литералов или `as const`; `erasableSyntaxOnly` в tsconfig
`does not provide an export named 'Item'`	Тип импортирован без `type`	Имя в ошибке — это тип	`import type { Item }` и `verbatimModuleSyntax`
`ERR_MODULE_NOT_FOUND` для своего файла	Импорт без расширения	Путь в ошибке без `.ts`	`import './item.ts'`
Вывод обрезан при передаче в `head`, `wc` или файл	`process.exit()` до завершения записи в канал	Медленный читатель: `| (sleep 1; wc -l)`	`process.exitCode`
`EXDEV: cross-device link not permitted`	Временный файл в `os.tmpdir()`, а целевой на другом разделе	Сравнить точки монтирования путей	Временный файл в папке целевого

Симптом	Причина	Как найти	Как исправить
Цена расходится на копейку	Дробные суммы, арифметика с плавающей точкой	Найти `parseFloat`, `/ 100`, `Math.round` в расчётах	Целые копейки, дробное отклонять
Процесс падает с кодом 1 и стеком отклонённого промиса	Промис без `await` и `.catch` или async-обработчик у `process.on`, `createServer`	`npm run lint`: `no-floating-promises`, `no-misused-promises`	`await`; `.catch` с обработкой; `void` — только если отказа быть не может
Процесс падает с необработанным событием `'error'`	У потока нет обработчика ошибок	Стек указывает на `Readable` или `Writable`	`pipeline` вместо `pipe`, ожидание через `once`
Остановка занимает секунды после SIGTERM	Keep-alive соединения после начатого запроса	Замерить время от сигнала до выхода с активным клиентом	`closeIdleConnections()` после каждого ответа при остановке
`EADDRINUSE`	Порт занят прошлым запуском	`lsof -i :3000`	Остановить прошлый процесс; в тестах `PORT=0`
Сервер запустился, но не отвечает на ожидаемом порту	`PORT=''` превратился в 0, и система выдала случайный порт	Лог с фактическим портом из `server.address()`	Проверять строку `PORT` регулярным выражением

Самопроверка

1. Почему сервер перестаёт отвечать на `/health`, пока выполняется синхронный пересчёт цен, хотя чтение файлов выполняет пул потоков libuv?
2. В каком порядке выполнятся `setTimeout(0)`, `setImmediate`, `process.nextTick` и `Promise.then`, заведённые в колбэке `fs.readFile`? Какую часть порядка нельзя считать гарантированной в главном ES-модуле?
3. Что вернёт `write()` при заполненном буфере, что произойдёт, если продолжить писать, и как `pipeline` решает эту проблему?
4. Зачем нужен `crlfDelay: Infinity` при чтении выгрузки?
5. Почему `const item = JSON.parse(line) as CatalogItem` ничего не проверяет, и как это исправить?
6. Почему дробную сумму в выгрузке нужно отклонить, а не округлить?
7. Почему временный файл создаётся в той же папке, что и целевой, и что даёт `flush: true`?

8. Чем отличаются исходы с кодами 1 и 3, и почему отсутствующий входной файл нельзя считать пустым каталогом?
9. Почему вместо `process.exit(1)` после вывода отчёта лучше `process.exitCode = 1`?
10. Что делает `server.close()` с keep-alive соединениями, и зачем примеру `closeIdleConnections()` после каждого ответа?
11. Что произойдёт, если в коде, который запускается через type stripping, объявить `enum`? Как узнать об этом до запуска?
12. Линтер сообщает `no-floating-promises` на строке `cache.refresh()` в обработчике запроса: кэш обновляется в фоне, ответ его не ждёт. Чем плохо поставить `void`, и как исправить?

Ответы — в приложении.

Вопросы собеседования

1. Опишите фазы цикла событий Node.js. Где выполняются колбэки `setImmediate`, а где — промисов?
2. Как найти в работающем сервисе код, который блокирует цикл событий?
3. Что такое обратное давление в потоках и как оно проявляется в продакшене?
4. Чем `pipeline` лучше цепочки `.pipe()`?
5. Как безопасно заменить файл, который читают другие процессы? Какие у этого способа границы?
6. Как хранить денежные суммы в JavaScript и почему не в `number` с дробной частью?
7. Что должно происходить в Node.js-сервисе между получением SIGTERM и выходом?
8. Чем проверка живости отличается от проверки готовности?

Что читать дальше

- Node.js: The Node.js Event Loop — https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick
- Node.js: Backpressuring in Streams — https://nodejs.org/en/learn/modules/backpressuring-in-streams
- Node.js API: Streams, раздел `stream.pipeline` — https://nodejs.org/api/stream.html
- Node.js API: Modules: TypeScript — https://nodejs.org/api/typescript.html
- Node.js API: process, разделы `process.exit` и «A note on process I/O» — https://nodejs.org/api/process.html
- Node.js API: File system, `fsPromises.writeFile` и опция `flush` — https://nodejs.org/api/fs.html
- Node.js API: HTTP, `server.close` и `server.closeIdleConnections` — https://nodejs.org/api/http.html
- Node.js API: Worker threads — https://nodejs.org/api/worker_threads.html

- POSIX: функция `rename` — https://pubs.opengroup.org/onlinepubs/9799919799/functions/rename.html
- RFC 9110, раздел 15.5.6 «405 Method Not Allowed» — https://www.rfc-editor.org/rfc/rfc9110#section-15.5.6
- Документация zod 4 — https://zod.dev
- typescript-eslint: правила `no-floating-promises` и `no-misused-promises` — https://typescript-eslint.io/rules/no-floating-promises/, https://typescript-eslint.io/rules/no-misused-promises/
- Node.js CLI: `--unhandled-rejections` — https://nodejs.org/api/cli.html#--unhandled-rejectionsmode

Глоссарий

Термин	Значение
Цикл событий (event loop)	Механизм libuv, который по фазам выполняет готовые колбэки в единственном JS-потоке
Микрозадача (microtask)	Колбэк промиса или `queueMicrotask`; очередь выполняется полностью после каждой операции
Блокировка цикла событий	Синхронная работа, во время которой процесс не обслуживает другие события
Поток (stream)	Объект, который отдаёт или принимает данные порциями
Обратное давление (backpressure)	Сигнал от медленного приёмника источнику остановиться, пока буфер не освободится
`highWaterMark`	Порог буфера потока, после которого `write()` возвращает `false`
JSONL	Формат «одна запись JSON на строку»
Валидация во время выполнения (runtime validation)	Проверка данных при работе программы, а не при компиляции
Минимальные денежные единицы	Копейки для рубля; суммы хранятся целым числом таких единиц
Атомарная замена	Замена файла, при которой читатель видит либо старую, либо новую версию целиком
Код выхода (exit code)	Число, которым процесс сообщает вызывающей стороне об исходе
Проверка живости (liveness check)	Ответ на вопрос «процесс жив и обслуживает события?»
Мягкая остановка (graceful shutdown)	Остановка, при которой начатые запросы завершаются, а новые не принимаются

Термин	Значение
Type stripping	Запуск TypeScript в Node.js удалением аннотаций типов без компиляции
Линтер (linter)	Программа, которая читает код, не запуская его, и находит места с вероятной ошибкой поведения
Висящий промис (floating promise)	Промис, результат которого никто не ждёт и отказ которого никто не обрабатывает
Необработанный отказ промиса (unhandled rejection)	Отклонённый промис без обработчика отказа; по умолчанию процесс завершается с кодом 1

Журнал изменений

Версия	Что изменилось
0.1.0	Первая версия
0.1.1	Таблица типовых ошибок помещается в область печати: PDF больше не уменьшается целиком
0.1.2	Титул: код примеров — папка examples/week-01 стартового репозитория; K19 и K24 в компетенциях недели; подраздел «Как работать с ИИ-ассистентом в курсе» и критерий о разделе «Помощь ИИ» в MR; честная граница предела 64 КБ; заглушки и публичные тесты приходят MR «Материалы недели 1»; упражнения входят в 7 часов реализации; без обещания worker_threads; проверка себя ждёт готовности сервера
0.2.0	Раздел «Линтер»: как читать ответ `no-floating-promises`, три исправления — `await`, `.catch` и `void`, правила `no-misused-promises`, `no-empty` и `preserve-caught-error`, когда допустим `eslint-disable`; `void` перед микрозадачей в примере порядка колбэков; `npm run check` — типы, линтер и тесты — в критериях и проверке себя; типовая ошибка о падении с кодом 1 от отклонённого промиса; вопрос самопроверки 12; термины «висящий промис» и «необработанный отказ промиса»; ссылки на правила typescript-eslint и `--unhandled-rejections`

Приложение. Ответы на самопроверку

1. Пул потоков выполняет только сам ввод-вывод. Колбэки с результатами, обработчики запросов и ваш расчёт выполняются в одном JS-потоке. Пока синхронный цикл пересчёта цен не закончился, цикл событий не может перейти к следующему колбэку: ответ на `/health` уже готов к обработке, но ждёт в очереди.

2. Внутри колбэка ввода-вывода: `nextTick`, затем `promise`, затем `setImmediate` (фаза check идёт сразу после poll), затем `setTimeout`. В главном ES-модуле не гарантированы две вещи. Порядок `setTimeout(0)` и `setImmediate` зависит от загрузки машины. Порядок `nextTick` и промиса зависит от версии Node.js и от наличия top-level `await`: в 22 и 24 первым идёт промис, в 26.12 без top-level `await` — `nextTick`.

3. `write()` вернёт `false`: буфер выше `highWaterMark`. Данные всё равно будут приняты и лягут в память. Если продолжать писать, буфер растёт до размера всех данных, и на большом файле процесс упадёт по памяти. `pipeline` перестаёт читать источник, пока приёмник не сообщит `drain`, а при ошибке любого звена закрывает все потоки и отклоняет промис.

4. В файлах из Windows строки разделены `\r\n`. По умолчанию readline считает `\r` и `\n` одним переводом строки, только если они пришли с промежутком меньше 100 мс. На границе порций чтения пауза возможна, и тогда появится лишняя пустая строка, а нумерация сдвинется. `Infinity` всегда склеивает `\r\n`.

5. `as` — указание компилятору, а не проверка. После стирания типов остаётся `const item = JSON.parse(line)`, и объект без `dailyPrice` или с ценой-строкой пройдёт дальше. Нужно принять результат как `unknown`, проверить схемой (`safeParse`) и использовать только проверенные данные, а тип вывести из схемы.

6. Копейки — минимальная единица; дробь копейки означает ошибку в источнике: рубли перепутаны с копейками или ячейку исправили вручную. Округление спрячет ошибку и даст неверную цену без следа. Отказ с номером строки покажет её тому, кто может исправить источник.

7. `rename` атомарно заменяет файл только в пределах одной файловой системы. Из другого раздела он упадёт с `EXDEV`, и атомарности не будет. `flush: true` вызывает `fsync` перед закрытием: данные дойдут до диска раньше, чем новое имя начнёт на них указывать. Без этого после сбоя питания можно получить целевой файл нулевой длины.

8. Код 1 — с вводом всё в порядке, но данные неверны; сотрудник исправляет файл по номерам строк. Код 3 — программа не смогла прочитать или записать файл; исправлять нужно окружение: путь, права, место на диске. Если считать отсутствующий файл пустым каталогом, импорт либо сотрёт рабочий каталог, либо сообщит об ошибке данных, которой нет.

9. При выводе в канал запись в stdout на Linux и macOS асинхронна. `process.exit()` завершает процесс сразу, и хвост отчёта теряется. `process.exitCode` только задаёт код, а процесс завершается, когда всё дописано.

10. Начиная с Node.js 19 `server.close()` перестаёт принимать соединения и закрывает те, что простаивают в момент вызова. Соединение запроса, который ещё выполнялся, после ответа становится простаивающим, но уже никто его не закрывает: оно живёт до `keepAliveTimeout` или пока клиент не закроет его сам. Вызов `closeIdleConnections()` на завершении каждого ответа во время остановки закрывает такие соединения сразу.

11. Node.js откажется выполнять файл с ошибкой `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX`: `enum` порождает JavaScript-код, а type stripping только удаляет типы. Узнать об этом до запуска позволяет `tsc` с `erasableSyntaxOnly` в `tsconfig.json`.

12. `void` скажет «отказа быть не может», а он возможен: Redis или база не ответят. Отказ останется необработанным и завершит процесс с кодом 1 посреди работы. Ждать обновления ответ действительно не должен, поэтому нужен `.catch` с обработкой: записать в журнал, что кэш не обновлён, и посчитать в метрике. Если обновление обязано закончиться до ответа — `await`.

© Евгений Надточеев. Все права защищены. Версия 0.2.0. Шрифты Inter и JetBrains Mono — SIL Open Font License 1.1.